Контест 5 · Конвейеры и секреты

Неделя 10 · темы недель 9–10: workflow, `needs` и матрица, версии, переменные окружения и секреты, деплой и откат. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Конвейер

Конвейер задан словарём «job → список needs». Пример для всех задач A:

```
J = {"lint": [], "types": [], "test": ["lint"], "build": ["test", "types"], "e2e":
["build"], "docs": []}
D = {"lint": 1, "types": 2, "test": 5, "build": 3, "e2e": 4, "docs": 2}      # минуты
```

A1. `stages(jobs)` — волны запуска: в волне все jobs, чьи `needs` выполнены в прошлых волнах; внутри волны — по имени. Цикл или ссылка на несуществующий job — `None`.

```
Ввод:  stages(J)
Вывод: [["docs", "lint", "types"], ["test"], ["build"], ["e2e"]]
```

A2. `pipeline_time(jobs, duration)` — сколько минут идёт весь конвейер, если машин сколько угодно.

```
Ввод:  pipeline_time(J, D)
Вывод: 13                    # lint 1 → test 5 → build 3 → e2e 4
```

A3. `affected(jobs, failed)` — какие jobs GitHub пропустит, если упал `failed`: все, кто прямо или через других от него зависит. По имени.

```
Ввод:  affected(J, "lint"), affected(J, "e2e")
Вывод: ["build", "e2e", "test"], []
```

A4*. `runners_time(jobs, duration, k)` — у вас `k` машин. Как только машина освободилась, она берёт готовый job с наименьшим именем. Сколько минут займёт конвейер?

```
Ввод:  runners_time(J, D, 1), runners_time(J, D, 2)
Вывод: 17, 13
```

B. Версии

B1. `sort_versions(vs)` — по возрастанию как версии, а не как строки. При равных числах более короткая запись раньше: `3.12` перед `3.12.0`.

```
Ввод:  sort_versions(["3.10", "3.9", "3.12.1", "3.12", "3.12.0", "2.7"])
Вывод: ["2.7", "3.9", "3.10", "3.12", "3.12.0", "3.12.1"]
```

B2. `satisfies(version, spec)` — подходит ли версия под условия через запятую: `>=`, `<=`, `>`, `<`, `==`, `!=`. Недостающие части — нули (`3.12` равно `3.12.0`). В `==` и `!=` можно `*` в конце: `==3.12.*`. Пустое условие — подходит всё. Другой оператор — `ValueError`.

```
Ввод:  satisfies("3.12.1", ">=3.10,<3.13"), satisfies("3.11.0", "!=3.11.*"),
satisfies("3.12", "==3.12.0")
Вывод: True, False, True
```

B3. `bump(version, part)` — следующая версия `major.minor.patch`: младшие части обнуляются.

```
Ввод:  bump("1.4.2", "major"), bump("1.4.2", "minor"), bump("0.9.9", "patch")
Вывод: "2.0.0", "1.5.0", "0.9.10"
```

C. Секреты и окружение

C1. `env_diff(old, new)` — что поменялось в окружении, **без значений** (их нельзя печатать в лог): `"+ KEY"` — добавлен, `"- KEY"` — удалён, `"~ KEY"` — изменён. По ключу.

```
Ввод:  env_diff({"A": "1", "B": "2", "C": "3"}, {"A": "1", "B": "x", "D": "4"})
Вывод: ["~ B", "- C", "+ D"]
```

C2. `interpolate(value, env)` — подстановка как в `docker compose`: `$VAR` и `${VAR}` (нет — пустая строка), `${VAR:-def}` (нет **или пусто** → `def`), `${VAR-def}` (только если нет), `$$` → `$`.

```
Ввод:  env = {"USER": "game", "EMPTY": "", "PORT": "8000"}
       interpolate("${HOST:-localhost}:${PORT}", env), interpolate("${EMPTY:-d}|${EMPTY-
d}|${NOPE-d}", env)
       interpolate("cost $$5 ${NOPE}x", env)
Вывод: "localhost:8000", "d||d", "cost $5 x"
```

C3. `redact_url(url)` — спрятать пароль в адресе подключения: `схема://пользователь:пароль@…`. Пользователь остаётся, пароля нет — строка без изменений. `@` после `/` — уже не про пароль.

```
Ввод:  redact_url("postgresql://game:S3cr3t@db:5432/game"),
redact_url("redis://:pw@redis:6379/0")
       redact_url("https://example.org/a@b")
Вывод: "postgresql://game:***@db:5432/game", "redis://:***@redis:6379/0",
"https://example.org/a@b"
```

C4*. `leaked_tokens(commits)` — проверка истории. Коммиты идут по порядку, у каждого `sha` и `files: {путь: (добавленные строки, удалённые строки)}`. Токен бота: 8–10 цифр, `:`, 35 символов `A-Za-z0-9_-`. Верните для каждого токена `(маска, sha первого появления, есть ли он сейчас хоть в одном файле)` в порядке появления. Маска — первые 4 и последние 2 символа через `…`.

```
Ввод:  T1 = "123456789:AAH" + "x" * 32
       [{"sha": "a1", "files": {"bot.py": (['TOKEN = "' + T1 + '"'], [])}},
        {"sha": "b2", "files": {"bot.py": (["TOKEN = os.getenv('BOT_TOKEN')"], ['TOKEN = "'
+ T1 + '"'])}}]
Вывод: [("1234…xx", "a1", False)]          # удалён, но в истории есть — токен надо отозвать
```

D. Деплой

D1. `next_release(tags, part="patch")` — следующий тег релиза после наибольшего корректного `vX.Y.Z` (сравнение как версий). Остальные теги пропускаем. Корректных нет — `"v0.1.0"`.

```
Ввод:  next_release(["v1.2.3", "v1.10.0", "release-2", "v1.9.9", "v2.0"]),
next_release(["v1.2.3"], "minor")
Вывод: "v1.10.1", "v1.3.0"
```

D2. `rollback_target(deploys)` — история деплоев `(версия, "ok" | "failed")`. Сейчас работает последняя `ok`. Откатываемся на ближайшую более раннюю `ok`, **отличную** от текущей. Нет такой — `None`.

```
Ввод:  rollback_target([("1.0", "ok"), ("1.1", "ok"), ("1.2", "failed"), ("1.1", "ok")])
Вывод: "1.0"
```

D3. `canary(errors, requests, threshold, window)` — новая версия получает часть трафика. По минутам известны ошибки и запросы. Найдите первую минуту (с нуля), когда доля ошибок за последние `window` минут (включая эту) **строго больше** `threshold`. В начале окно неполное — берём, что есть; запросов 0 — доля 0. Не было — `None`.

```
Ввод:  canary([0, 1, 5, 9, 2], [100] * 5, 0.03, 2)
Вывод: 3                    # минута 2: 6/200 = 0.03 — ещё нет; минута 3: 14/200 = 0.07
```

E. Проект и алгоритмы

E1. Зелёный CI в репозитории команды: `lint` → `test` → `build`, защита `main` (обязательные проверки и одно одобрение). Покажите PR, где CI сначала красный, потом зелёный, и попытку пуша в `main` напрямую.

E2. Автодеплой: после merge в `main` workflow выкладывает новую версию на сервер (staging), версия видна по `/health`. Покажите откат на прошлую версию одной командой или повторным запуском workflow.

E3. Секреты: в `AGENTS.md` есть раздел о секретах, в репозитории есть `.env.example`, нет `.env`. Хук или шаг CI со сканером (лаба 10 или `gitleaks`) ловит токен в тестовом коммите. Объясните, что делать, если токен уже запушен.

E4 · босс. Обновление без простоя: `n` копий сервиса за Nginx. За каждую часть свой плюс.

а) `rolling_plan(n, max_unavailable, max_surge)` — состояния `(старых, новых)` после каждого шага, пока все `n` не новые. Шаг: сначала запускаем новые, сколько можно (всего копий не больше `n + max_surge`, новых не больше `n`), затем гасим старые, пока работающих не меньше `n − max_unavailable`. Оба параметра 0 — `ValueError`.

```
Ввод:  rolling_plan(4, 1, 1), rolling_plan(3, 1, 0)
Вывод: [(2, 1), (0, 3), (0, 4)], [(2, 0), (1, 1), (0, 2), (0, 3)]
```

б) `ready_ticks(checks, need)` — у каждой новой копии список результатов проверок `/health`. Копия готова после `need` успешных проверок **подряд**. Для каждой — номер проверки (с нуля), после которой она готова, или `None`.

```
Ввод:  ready_ticks([[True, True, False, True, True, True], [False, True, True], [True,
False]], 2)
Вывод: [1, 2, None]
```

в) `rollback_plan(n, max_unavailable, max_surge, state)` — новая версия сломалась в состоянии `state`. Откат по тем же правилам, только теперь «новые» — это старые. Ответ — снова в виде `(старых, новых)`.

```
Ввод:  rollback_plan(3, 0, 1, (1, 2)), rollback_plan(4, 1, 1, (2, 1))
Вывод: [(2, 1), (3, 0)], [(4, 0)]
```

E5*. `rebuilt_from(dockerfile, changed, base_changed=False)` — индекс первой инструкции Dockerfile, которая пересоберётся (после неё пересобирается всё); всё из кэша — `None`. `FROM` пересобирается, только если поменялся базовый образ. `COPY` / `ADD`: если какой-то изменённый файл подходит под один из источников (`fnmatch`; папка — всё внутри неё; `.` — всё). Флаги вида `--chown=…` — не источники. Остальные инструкции сами по себе из кэша.

```
Ввод:  DF = ["FROM python:3.12-slim", "WORKDIR /app", "COPY requirements.txt .",
             "RUN pip install -r requirements.txt", "COPY app/ ./app", 'CMD ["python", "-m",
"app"]']
       rebuilt_from(DF, ["app/main.py"]), rebuilt_from(DF, ["requirements.txt"]),
rebuilt_from(DF, ["README.md"])
Вывод: 4, 2, None            # поэтому requirements.txt копируют раньше кода
```